

Tema 0.

Regex: expresiones regulares en Python

Aprender a reconocer patrones en el texto

Material teórico | Programación para PLN | Universidad Pontificia Comillas

Índice de contenidos

Contexto profesional.....	3
Encargo: preparar el kit de detección de patrones.....	3
El caso de trabajo.....	3
Qué vas a aprender en esta unidad.....	3
Mapa conceptual de la unidad.....	4
1. Marco teórico.....	4
1.1. Qué son las expresiones regulares: de la teoría de autómatas a la práctica textual.....	4
1.2. Regex y PLN: relevancia para quien trabaja con lenguaje.....	5
1.3. El módulo re de Python: motor de expresiones regulares.....	5
1.4. re.search(): localizar la primera coincidencia.....	5
1.5. El objeto Match y el valor None: cómo Python representa «no encontrado».....	5
1.6. Metacaracteres básicos: el punto, \w, \s y \d.....	5
1.7. Cuantificadores + y *: la repetición como propiedad del patrón.....	5
1.8. Límites de palabra \b: precisión en la coincidencia.....	6
1.9. re.findall(): extraer todas las coincidencias de un patrón.....	6
1.10. re.sub(): sustitución basada en patrones.....	6
1.11. Clases de caracteres negadas: [^\w\s] para limpieza de puntuación.....	6
1.12. re.match() y el ancla ^: coincidir solo al principio de la cadena.....	6
1.13. Cuantificadores de repetición exacta: {n} y {n,}.....	6
1.14. De la sintaxis a la interpretación lingüística: qué puede y qué no puede una regex.....	6
2. Python para PLN: de la teoría al primer patrón con LinguaAdapt.....	7
2.1. Importar el módulo re.....	7
2.2. Primera búsqueda: re.search() y el valor None.....	7
2.3. Comprobar una coincidencia con una condición if.....	7
2.4. Palabras que empiezan por una letra concreta: \b y \w+.....	7
2.5. Sustituir apariciones con re.sub().....	8
2.6. Limpiar puntuación con una clase negada.....	8
2.7. Contar y listar coincidencias con re.findall().....	8
2.8. Detectar patrones estructurados: direcciones de correo electrónico.....	8
2.9. Palabras que terminan en un sufijo: verbos en infinitivo.....	9
2.10. Contar palabras de una frase con límites de palabra.....	9
2.11. Detectar mayúsculas iniciales: nombres propios y comienzos de oración.....	9

2.12. Anclar al inicio de la cadena: re.match() y el símbolo ^.....	10
2.13. Extraer números: de \d+ a cuantificadores de longitud exacta.....	10
2.14. Eliminar números de un texto con re.sub().....	11
2.15. Tabla resumen de sintaxis regex.....	11
Errores frecuentes de aprendizaje.....	11
Síntesis final.....	12
Cierre narrativo.....	12
Referencias.....	12

Contexto profesional

En cualquier proyecto real de procesamiento del lenguaje natural —ya se trabaje en una agencia de traducción y localización, en un equipo de análisis de datos lingüísticos o en un departamento de control de calidad de contenidos— antes de poder analizar un texto en profundidad hace falta saber localizar, limpiar y extraer fragmentos concretos dentro de él. Esa es la capacidad que trabaja esta unidad: reconocer y manipular patrones dentro de un texto sin tener que leerlo entero a mano, línea por línea. Esa herramienta son las expresiones regulares, conocidas habitualmente por su nombre en inglés, *regex*.

En cualquier proyecto de procesamiento del lenguaje natural, buena parte del trabajo previo consiste en localizar, limpiar o extraer fragmentos concretos de un texto: una palabra que se repite, una dirección de correo electrónico, un número de teléfono, una fecha o una secuencia de caracteres que sigue una forma reconocible. Hacerlo letra por letra sería inviable en corpus grandes. *Regex* ofrece un lenguaje formal y compacto para describir esos patrones y dejar que sea el propio ordenador quien los busque.

Objetivo de la unidad: preparar el *kit* de detección de patrones

Esta unidad no te pide todavía que midas nada, como sí ocurrirá más adelante, en el Tema 1, con tu primer encargo profesional en *LinguaAdapt*. Te pide, antes de eso, que aprendas a mirar un texto de otra manera: no como una sucesión de palabras que se leen de corrido, sino como una cadena de caracteres en la que se pueden buscar formas recurrentes. Este tema es, por tanto, una unidad de preparación: el resto de las unidades de la asignatura dará por hecho que sabes construir y aplicar patrones sencillos con el módulo `re` de Python.

El caso de trabajo

Estas son las capacidades mínimas que debes dominar antes de continuar:

- Comprobar si una palabra concreta aparece en un texto.
- Sustituir una palabra por otra en todas sus apariciones.
- Eliminar la puntuación de un texto sin reescribirlo a mano.
- Contar y listar todas las apariciones de un mismo patrón.
- Reconocer estructuras con forma reconocible, como una dirección de correo electrónico.
- Detectar palabras que cumplen una regularidad morfológica, como los verbos en infinitivo terminados en «-ar».
- Extraer y eliminar secuencias numéricas de un texto.

Detrás de esta lista hay una idea que atraviesa toda la unidad: muchas tareas lingüísticas que parecen requerir «comprender» un texto pueden resolverse, al menos de forma aproximada, con reglas formales sobre la superficie de la cadena de caracteres. *Regex* es la puerta de entrada a ese tipo de razonamiento, y el paso previo indispensable antes de abordar tareas de PLN más complejas.

Qué vas a aprender en esta unidad

Al terminar esta guía, deberías poder explicar y aplicar los siguientes conocimientos:

1. Qué es una expresión regular y qué papel ocupa dentro de la jerarquía de lenguajes formales.
2. Cómo importar y utilizar el módulo `re` de la biblioteca estándar de Python.

3. Cómo usar `re.search()` para localizar la primera coincidencia de un patrón.
4. Cómo interpretar el objeto `Match` y el valor `None` que puede devolver una búsqueda.
5. Qué significan los metacaracteres básicos: el punto, `\w`, `\s` y `\d`.
6. Cómo usar los cuantificadores `+` y `*` para expresar repetición.
7. Qué aporta el límite de palabra `\b` a la precisión de una coincidencia.
8. Cómo usar `re.findall()` para extraer todas las coincidencias de un patrón.
9. Cómo usar `re.sub()` para sustituir o eliminar texto según un patrón.
10. Cómo anclar una búsqueda al principio de la cadena con `re.match()` y el símbolo `^`.
11. Cómo controlar la longitud exacta de una coincidencia con cuantificadores como `{4}` o `{4,}`.
12. Cómo interpretar con criterio lingüístico los resultados de una búsqueda basada en patrones, reconociendo sus límites.

Mapa conceptual de la unidad

Antes de entrar en el detalle, conviene fijar el mapa general de la unidad: un patrón se escribe con una sintaxis formal y se aplica a un texto a través de una de las funciones del módulo `re`; cada función devuelve un tipo de resultado distinto, que hay que saber interpretar.

- Patrón (regex): secuencia formal escrita con metacaracteres, cuantificadores y clases de caracteres.
- `re.search(patrón, texto)`: localiza la primera coincidencia → devuelve un objeto `Match` o `None`.
- `re.match(patrón, texto)`: igual que `search()`, pero solo si la coincidencia empieza en la primera posición del texto.
- `re.findall(patrón, texto)`: localiza todas las coincidencias → devuelve una lista de cadenas.
- `re.sub(patrón, reemplazo, texto)`: sustituye las coincidencias → devuelve una cadena nueva.
- Piezas de sintaxis: metacaracteres (`.`, `\w`, `\s`, `\d`), cuantificadores (`+`, `*`, `{n}`, `{n,}`), límites (`\b`, `^`, `$`) y clases negadas (`[^...]`).

1. Marco teórico

Esta unidad introduce la primera herramienta formal de la asignatura para trabajar con texto: la expresión regular. El objetivo del marco teórico es explicar de dónde procede esta herramienta, por qué es relevante para el procesamiento del lenguaje natural y cómo interpretar cada uno de los elementos de sintaxis antes de aplicarlos en el cuaderno de trabajo.

Al final de este marco teórico deberías ser capaz de leer un patrón regex sencillo y anticipar, sin ejecutarlo, qué tipo de fragmentos de texto localizaría.

1.1. Qué son las expresiones regulares: de la teoría de autómatas a la práctica textual

Una expresión regular es una notación formal para describir conjuntos de cadenas de caracteres mediante un patrón. Formalmente, los lenguajes que pueden describirse con expresiones regulares corresponden a los lenguajes regulares de la jerarquía de Chomsky, la clase más restrictiva pero también la más eficiente de reconocer computacionalmente (Chomsky, 1956). Cualquier expresión

regular puede convertirse en un autómata finito equivalente capaz de decidir, en tiempo lineal, si una cadena pertenece o no al patrón descrito (Hopcroft *et al.*, 2006).

Esta base teórica explica por qué las expresiones regulares son tan rápidas y fiables para tareas de búsqueda: no necesitan «entender» el texto, solo recorrer la cadena de caracteres aplicando una regla mecánica. Esa misma propiedad es, como se verá en la unidad, también su principal limitación lingüística.

1.2. Regex y PLN: relevancia para quien trabaja con lenguaje

En procesamiento del lenguaje natural, buena parte del trabajo de preprocesamiento —limpiar texto, normalizar puntuación, separar *tokens*, detectar estructuras recurrentes— se apoya en expresiones regulares antes incluso de aplicar técnicas estadísticas o modelos de aprendizaje automático (Bird *et al.*, 2009; Manning *et al.*, 2008). Jurafsky y Martin (2026) sitúan las expresiones regulares entre las primeras herramientas que debe dominar cualquier persona que trabaje con texto de forma computacional, precisamente porque son la vía más directa para convertir una intuición lingüística sobre la forma de las palabras en una regla ejecutable.

Para una lingüista o un lingüista computacional, esto significa que muchas preguntas del tipo «¿qué palabras del texto tienen esta forma?» pueden responderse sin recurrir todavía a herramientas de análisis morfosintáctico avanzado: basta con describir la forma como un patrón.

1.3. El módulo `re` de Python: motor de expresiones regulares

Python incluye en su biblioteca estándar el módulo `re`, que implementa un motor de expresiones regulares y expone funciones para buscar, extraer y sustituir patrones dentro de cadenas de texto (Python Software Foundation, 2026). No hay que instalar nada adicional: basta con importar el módulo al principio del cuaderno.

1.4. `re.search()`: localizar la primera coincidencia

`re.search(patrón, texto)` recorre el texto buscando la primera parte que coincida con el patrón. En cuanto la encuentra, se detiene y la devuelve envuelta en un objeto especial; si recorre todo el texto sin encontrar ninguna coincidencia, devuelve el valor `None`.

1.5. El objeto `Match` y el valor `None`: cómo Python representa «no encontrado»

Cuando `re.search()` tiene éxito, devuelve un objeto `Match`, que además de indicar que hubo coincidencia guarda información como la posición exacta (*span*) y el fragmento de texto encontrado (*match*). Cuando no hay coincidencia, el resultado es `None`, el valor que Python reserva para representar la ausencia de un objeto. Distinguir un objeto `Match` de `None` es la base de cualquier condición del tipo «si se encontró el patrón, haz esto».

1.6. Metacaracteres básicos: el punto, `\w`, `\s` y `\d`

Los metacaracteres son símbolos que, dentro de un patrón, no representan un carácter literal, sino una categoría de caracteres. El punto (`.`) representa cualquier carácter salvo un salto de línea; `\w` representa letras, dígitos o el guion bajo; `\s` representa un espacio en blanco (incluyendo tabulaciones y saltos de línea); `\d` representa un dígito del 0 al 9. Estos cuatro metacaracteres son la base sobre la que se construyen patrones más complejos.

1.7. Cuantificadores + y *: la repetición como propiedad del patrón

Los cuantificadores indican cuántas veces debe repetirse el elemento anterior del patrón. El símbolo + exige una o más repeticiones; el símbolo * admite cero o más. Por ejemplo, `\w+` describe una secuencia de una o más letras, dígitos o guiones bajos consecutivos, es decir, algo muy próximo a lo que entendemos por «una palabra» desde un punto de vista puramente formal.

1.8. Límites de palabra `\b`: precisión en la coincidencia

`\b` marca un límite de palabra: la frontera entre un carácter de tipo `\w` y uno que no lo es (o el principio o el final de la cadena). Añadir `\b` a los extremos de un patrón evita coincidencias parciales dentro de palabras más largas. Sin ese límite, un patrón como `\w+ar` podría coincidir también con el final de una palabra que no es un verbo en infinitivo, simplemente porque termina de forma parecida.

1.9. `re.findall()`: extraer todas las coincidencias de un patrón

A diferencia de `re.search()`, que se detiene en la primera coincidencia, `re.findall(patrón, texto)` recorre todo el texto y devuelve una lista con todas las coincidencias encontradas. Es la función más habitual cuando el objetivo no es comprobar si algo aparece, sino contar o extraer todas las apariciones de un patrón.

1.10. `re.sub()`: sustitución basada en patrones

`re.sub(patrón, reemplazo, texto)` busca todas las coincidencias de un patrón y las sustituye por el texto de reemplazo indicado, devolviendo una cadena nueva. Si el reemplazo es una cadena vacía, el efecto es eliminar del texto todo lo que coincide con el patrón.

1.11. Clases de caracteres negadas: `[^\w\s]` para limpieza de puntuación

Los corchetes definen una clase de caracteres: un conjunto de caracteres cualquiera de los cuales cuenta como coincidencia. Cuando el primer carácter dentro de los corchetes es el símbolo ^, la clase se convierte en una negación: coincide con cualquier carácter que no pertenezca al conjunto. El patrón `[^\w\s]` describe, por tanto, «cualquier carácter que no sea una letra, un dígito, un guion bajo ni un espacio», que en la práctica corresponde casi siempre a los signos de puntuación.

1.12. `re.match()` y el ancla ^: coincidir solo al principio de la cadena

`re.match(patrón, texto)` se comporta como `re.search()`, pero solo tiene éxito si la coincidencia empieza justo en la primera posición del texto. El símbolo ^ dentro de un patrón cumple una función equivalente cuando se usa junto a `re.match()`: ancla la búsqueda al principio de la cadena, de modo que el patrón solo coincide si el texto empieza exactamente de esa forma.

1.13. Cuantificadores de repetición exacta: `{n}` y `{n,}`

Además de + y *, es posible fijar un número exacto o mínimo de repeticiones entre llaves. `{4}` exige exactamente cuatro repeticiones consecutivas del elemento anterior; `{4,}` exige cuatro o más. Estos cuantificadores permiten controlar con precisión la longitud de una coincidencia, algo especialmente útil al trabajar con secuencias numéricas de longitud variable, como números de teléfono o códigos.

1.14. De la sintaxis a la interpretación lingüística: qué puede y qué no puede una regex

Una expresión regular aplica una regla puramente formal sobre la superficie de la cadena de caracteres: no «entiende» el idioma, no distingue un nombre propio de un nombre común por su significado, y no reconoce una excepción morfológica a menos que se le indique explícitamente en el patrón. Por eso, un patrón que funciona bien sobre un ejemplo concreto puede fallar en textos reales con mayor variedad léxica, abreviaturas o casos especiales.

Esta unidad, por tanto, no enseña solo una sintaxis. Enseña una primera forma de responsabilidad técnica: cada patrón debe diseñarse pensando en qué casos reales puede cubrir y en cuáles necesitará ajustes o revisión humana posterior. Esa misma actitud crítica será necesaria en todas las unidades siguientes de la asignatura.

2. Python para PLN: de la teoría a la práctica con el módulo `re`

Esta sección recorre, paso a paso, el cuaderno principal de trabajo, mostrando el código exactamente como aparece en él junto con el resultado que produce.

2.1. Importar el módulo `re`

Todo el trabajo de esta unidad empieza importando el módulo `re` de la biblioteca estándar de Python:

```
import re
```

A partir de esta línea, el cuaderno tiene disponibles todas las funciones de expresiones regulares que se usarán a lo largo de la unidad.

2.2. Primera búsqueda: `re.search()` y el valor `None`

El primer ejemplo busca la palabra «gato» dentro de un texto:

```
texto = "Mi gato duerme todo el día."
resultado = re.search("gato", texto)
print(resultado)
<re.Match object; span=(3, 7), match='gato'>
```

`re.search()` ha encontrado la palabra «gato» en las posiciones 3 a 7 del texto y ha devuelto un objeto `Match` con esa información. Si repetimos la búsqueda sobre el mismo texto pero cambiando la palabra buscada por «perro»:

```
texto = "Mi gato duerme todo el día."
resultado = re.search("perro", texto)
print(resultado)
None
```

El resultado es `None`: el patrón «perro» no aparece en el texto, así que `re.search()` no tiene nada que devolver.

2.3. Comprobar una coincidencia con una condición `if`

El valor devuelto por `re.search()` se puede usar directamente en una condición, porque un objeto `Match` se evalúa como verdadero y `None` se evalúa como falso:

```
if resultado:
    print(";Encontré la palabra 'gato'!")
```



```
else:
    print("No encontré la palabra 'gato'.")
No encontré la palabra 'gato'.
```

Como en este punto resultado contiene el `None` obtenido al buscar «perro», la condición toma la rama `else`. Este patrón —guardar el resultado de `re.search()` en una variable y comprobarlo después con un `if`— es la forma más habitual de usar expresiones regulares en un programa real.

2.4. Palabras que empiezan por una letra concreta: `\b` y `\w+`

El siguiente ejemplo busca todas las palabras de un texto que empiezan por la letra «c»:

```
texto2 = "El coche y la casa están cerca del campo."
palabras_c = re.findall(r"\bc\w+", texto2)
print("Palabras que empiezan por 'c':", palabras_c)
Palabras que empiezan por 'c': ['coche', 'casa', 'cerca', 'campo']
```

El patrón `\bc\w+` se lee así: un límite de palabra, seguido de una «c» literal, seguida de una o más letras, dígitos o guiones bajos. El límite de palabra `\b` es lo que impide que el patrón coincida con una «c» que aparezca en mitad de otra palabra.

2.5. Sustituir apariciones con `re.sub()`

El método `re.sub()` permite reemplazar todas las apariciones de un patrón por otro texto:

```
texto3 = "María tiene un perro. El perro se llama Max."
nuevo_texto = re.sub("perro", "gato", texto3)
print(nuevo_texto)
María tiene un gato. El gato se llama Max.
```

Las dos apariciones de «perro» se han sustituido por «gato». `re.sub()` no se detiene en la primera coincidencia: actúa sobre todas las que encuentra en el texto.

2.6. Limpiar puntuación con una clase negada

Para eliminar los signos de puntuación de un texto sin reescribirlo a mano, basta con sustituir por una cadena vacía todo carácter que no sea letra, dígito, guion bajo o espacio:

```
texto4 = ";Hola! ¿Cómo estás? Estoy bien, gracias."
texto_limpio = re.sub(r"[^\w\s]", "", texto4)
print("Texto sin puntuación:", texto_limpio)
Texto sin puntuación: Hola Cómo estás Estoy bien gracias
```

El patrón `[^\w\s]` coincide con cualquier signo de puntuación (¡, !, ¿, ?, la coma...) y `re.sub()` los elimina uno a uno, dejando intactas las letras, los dígitos y los espacios.

2.7. Contar y listar coincidencias con `re.findall()`

`re.findall()` permite tanto listar como contar las apariciones de un patrón, combinándolo con `len()`:

```
texto5 = "El gato está en el tejado. El gato duerme."
apariciones = re.findall("gato", texto5)
```

```
print("Número de veces que aparece 'gato':", len(apariciones))
print("Lista de coincidencias:", apariciones)
Número de veces que aparece 'gato': 2
Lista de coincidencias: ['gato', 'gato']
```

La palabra «gato» aparece dos veces en el texto, y la lista devuelta por `re.findall()` refleja exactamente esas dos coincidencias.

2.8. Detectar patrones estructurados: direcciones de correo electrónico

Las expresiones regulares también sirven para reconocer estructuras con una forma característica, como una dirección de correo electrónico:

```
texto6 = "Puedes escribir a juan@example.com o a maria@gmail.com."
correos = re.findall(r"\w+@\w+\.com", texto6)
print("Correos encontrados:", correos)
Correos encontrados: ['juan@example.com', 'maria@gmail.com']
```

El patrón `\w+@\w+\.com` describe una secuencia de caracteres alfanuméricos, seguida de una arroba, seguida de otra secuencia alfanumérica y terminada en «.com». Es un patrón simplificado con fines didácticos: no cubre todos los dominios ni todos los formatos válidos de correo electrónico, pero ilustra la lógica de reconocer una estructura por su forma.

2.9. Palabras que terminan en un sufijo: verbos en infinitivo

El siguiente patrón localiza palabras que terminan en «-ar», como ocurre con muchos verbos en infinitivo del español:

```
texto = "Cantar, bailar, comer y vivir son verbos comunes."
palabras_ar = re.findall(r"\b\w+ar\b", texto)
print("Palabras que terminan en 'ar':", palabras_ar)
Palabras que terminan en 'ar': ['Cantar', 'bailar']
```

Solo «Cantar» y «bailar» coinciden con el patrón, porque son las únicas palabras del texto que terminan exactamente en «ar» delimitado por dos límites de palabra. «Comer» y «vivir» no coinciden, aunque también sean verbos en infinitivo, porque no terminan en esa secuencia concreta: el patrón detecta una forma, no una categoría gramatical.

2.10. Contar palabras de una frase con límites de palabra

El patrón `\b\w+\b` permite extraer y contar las palabras de una frase:

```
frase = "La lingüística computacional es fascinante."
palabras = re.findall(r"\b\w+\b", frase)
print("Palabras encontradas:", palabras)
print("Número total de palabras:", len(palabras))
Palabras encontradas: ['La', 'lingüística', 'computacional', 'es', 'fascinante']
Número total de palabras: 5
```

A diferencia de `.split()`, que solo separa por espacios, este patrón define explícitamente qué cuenta como palabra: una secuencia de caracteres alfanuméricos delimitada por límites de palabra.

2.11. Detectar mayúsculas iniciales: nombres propios y comienzos de oración

El patrón `\b[A-Z]\w+` localiza palabras que empiezan por una letra mayúscula:

```
texto = "Madrid es la capital de España. Juan estudia Lenguas Modernas."

mayusculas = re.findall(r"\b[A-Z]\w+", texto)

print("Palabras con inicial mayúscula:", mayusculas)
Palabras con inicial mayúscula: ['Madrid', 'España', 'Juan', 'Lenguas', 'Modernas']
```

El resultado mezcla nombres propios («Madrid», «España», «Juan») con una palabra que solo tiene mayúscula inicial por ir a principio de oración («Lenguas», dentro de «Lenguas Modernas»). Este ejemplo es una buena muestra de los límites de la técnica: detectar mayúsculas iniciales es un indicio útil para localizar nombres propios, pero no una prueba concluyente, porque el patrón no distingue el motivo por el que una palabra empieza en mayúscula.

2.12. Anclar al inicio de la cadena: `re.match()` y el símbolo `^`

`re.match()` comprueba si el texto empieza exactamente por el patrón indicado:

```
frase = "Hola, ¿cómo estás?"

if re.match(r"^Hola", frase):
    print("La frase empieza por 'Hola'")
else:
    print("La frase NO empieza por 'Hola'")
La frase empieza por 'Hola'
```

Aunque `re.match()` ya ancla la búsqueda al principio de la cadena por sí solo, añadir el símbolo `^` dentro del patrón refuerza esa misma idea de manera explícita. Si la frase empezara por cualquier otra palabra, la condición tomaría la rama `else`.

2.13. Extraer números: de `\d+` a cuantificadores de longitud exacta

El patrón `\d+` extrae cualquier secuencia de uno o más dígitos consecutivos:

```
texto = "Tengo 2 gatos, 1 perro y 3 peces."

numeros = re.findall(r"\d+", texto)

print("Números encontrados:", numeros)
Números encontrados: ['2', '1', '3']
```

Si el texto incluye además un número de teléfono, un cuantificador mínimo como `{4,}` permite quedarse solo con las secuencias largas:

```
texto = "Tengo 2 gatos, 1 perro y 3 peces. Mi teléfono es el 679483750."

numeros = re.findall(r"\d{4,}", texto)

print("Números encontrados:", numeros)
Números encontrados: ['679483750']
```

Pero un cuantificador exacto sin límites de palabra puede fragmentar una secuencia larga en trozos de la longitud indicada, en lugar de respetarla como una unidad:

```
texto = ("Tengo 2 gatos, 1 perro y 3 peces. Mi teléfono es el "
        "679483750 y mi pin 1234.")

numeros_prueba1 = re.findall(r"\d{4}", texto)

print("Números encontrados en la prueba 1:", numeros_prueba1)
Números encontrados en la prueba 1: ['6794', '8375', '1234']
```

El patrón `\d{4}` exige grupos de exactamente cuatro dígitos, así que el número de teléfono de nueve cifras se ha dividido en fragmentos de cuatro en cuatro, y el pin de cuatro cifras coincide de forma completa por coincidencia de longitud. Añadiendo límites de palabra se corrige el problema:

```
numeros_prueba2 = re.findall(r"\b\d{4}\b", texto)

print("Números encontrados en la prueba 2:", numeros_prueba2)
Números encontrados en la prueba 2: ['1234']
```

Con `\b` a ambos lados, el patrón solo coincide con secuencias de exactamente cuatro dígitos que además estén delimitadas como unidad independiente. El número de teléfono, al tener nueve cifras seguidas sin ningún límite de palabra en medio, deja de coincidir, y solo se recupera el pin de cuatro dígitos. Este ejemplo es especialmente útil para entender por qué un cuantificador de longitud exacta necesita casi siempre ir acompañado de límites de palabra.

2.14. Eliminar números de un texto con `re.sub()`

Por último, `re.sub()` permite eliminar del texto cualquier secuencia numérica:

```
texto = ("Tengo 2 gatos, 1 perro y 3 peces. Mi teléfono es el "
        "679483750 y mi pin 1234.")

texto_sin_numeros = re.sub(r"\d+", "", texto)

print("Texto sin números:", texto_sin_numeros)
Texto sin números: Tengo  gatos,  perro y  peces. Mi teléfono es el  y mi pin .
```

Cada secuencia de dígitos se sustituye por una cadena vacía, dejando el resto del texto intacto (incluidos los dobles espacios que quedan donde antes había un número, una consecuencia habitual de este tipo de limpieza que conviene tener presente si el texto resultante se va a analizar después).

2.15. Tabla resumen de sintaxis regex

La siguiente tabla recoge, a modo de referencia rápida, los elementos de sintaxis trabajados en esta unidad:

Expresión	Significado
.	Cualquier carácter (excepto salto de línea)
\w	Letras, números o guion bajo
\s	Espacio en blanco
\d	Dígito (0-9)
+	Uno o más
*	Cero o más
\b	Límite de palabra
[]	Conjunto de caracteres
^	Comienzo de línea o negación en corchetes
\$	Final de línea

Errores frecuentes de aprendizaje

Olvidar que `re.search()` puede devolver `None`.

Si el patrón no aparece en el texto, resultado vale `None`. Usar ese resultado directamente (por ejemplo, `resultado.group()`) sin comprobar antes si es `None` provoca un error.

Confundir `\w` con «palabra» en sentido lingüístico.

`\w` describe letras, dígitos y el guion bajo. No distingue palabras reales de secuencias sin sentido lingüístico, ni entiende la morfología de la lengua.

Olvidar el límite de palabra `\b` y obtener coincidencias parciales.

Sin `\b`, un patrón como `\w+ar` puede coincidir con el final de una palabra que no es un verbo en infinitivo, simplemente porque termina de forma parecida.

Confundir `{4}` con «cuatro o más».

`{4}` exige exactamente cuatro repeticiones. Para expresar «cuatro o más» hace falta escribir `{4,}`, con la coma.

Pensar que una regex «entiende» el idioma.

Una expresión regular aplica una regla formal sobre caracteres. No reconoce significado, ni categoría gramatical, ni excepciones que no estén explícitas en el patrón.

Síntesis final

En esta unidad has aprendido a describir patrones formales sobre texto y a aplicarlos con el módulo `re` de Python: buscar la primera coincidencia con `re.search()`, anclarla al inicio de la cadena con `re.match()`, extraer todas las coincidencias con `re.findall()` y sustituir o eliminar texto con `re.sub()`. Has trabajado también con los elementos de sintaxis que hacen posible describir esos patrones: metacaracteres, cuantificadores, límites de palabra y clases de caracteres negadas.

La idea central que atraviesa la unidad es que muchas tareas de limpieza y extracción de texto pueden resolverse con reglas formales sobre la forma de las cadenas de caracteres, sin necesidad todavía de herramientas lingüísticas más sofisticadas. Esa misma idea, y esas mismas funciones, reaparecerán en las unidades siguientes de la asignatura cada vez que haga falta preparar o limpiar un texto antes de analizarlo.

Cierre narrativo

Con esta unidad completada, ya dispones del primer instrumento de tu *kit* de herramientas como futura lingüista computacional: la capacidad de reconocer patrones en un texto sin necesidad de leerlo entero a mano. No es un instrumento que sustituya el criterio lingüístico, sino uno que lo hace más eficiente: una regex bien diseñada libera tiempo para dedicarlo a las preguntas que sí requieren interpretación humana.

A partir de aquí, cada vez que en una unidad posterior necesites limpiar un texto, detectar una estructura recurrente o preparar un corpus antes de analizarlo, sabrás que ese primer paso puede

resolverse, al menos parcialmente, con una expresión regular bien construida. De hecho, ya en el Tema 1 pondrás en práctica esta misma idea en tu primer encargo profesional en LinguaAdapt.

Referencias

- Bird, S., Klein, E. y Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media. <https://www.nltk.org/book/>
- Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3), 113-124. <https://doi.org/10.1109/TIT.1956.1056813>
- Friedl, J. E. F. (2006). *Mastering regular expressions* (3.^a ed.). O'Reilly Media.
- Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2006). *Introduction to automata theory, languages, and computation* (3.^a ed.). Addison-Wesley.
- Jurafsky, D. y Martin, J. H. (2026). *Speech and language processing* (3rd ed. draft). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
- Manning, C. D., Raghavan, P. y Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. <https://nlp.stanford.edu/IR-book/>
- Python Software Foundation. (2026). re: Regular expression operations. *Python 3.14 documentation*. <https://docs.python.org/3/library/re.html>